数据结构关键笔记（C 指针操作口诀版）

体例约定（与《链表核心操作（只用 pq 两指针）》一致）：
*① 每节先写**指针/变量约定**（谁定位、谁新建、谁是前驱）；*
*② 每个核心操作只写**最少几行代码**，代码块外的↑行注释说明这行干什么；*
*③ 每个操作写明**初始条件**（指针一开始指向哪、怎么爬到位的）；*
*④ 同类的操作给**对比表**；*
*⑤ 每节末尾**核心记忆点**（几句话，背下来）。*

一、顺序表 SqList

约定：基址 `L->elem`（动态数组），`length` 当前长度，`size` 容量。插入删除的本质就是"搬元素"。

1.1 在第 i 位插入（先扩容检查，后移一位，插入）

初始：合法性检查（$1 \le i \le length+1$）；q 指向插入位。

```
    q = L->elem + i - 1;                                    // q = 插入位
    for (p = L->elem + L->length - 1; p >= q; --p) *(p+1) = *p;  // 从后往前逐个后移
    *q = e; ++L->length;                                    // 放入新元素，长度加 1
```

↑ 后移必须**从后往前**（从前往后会覆盖没搬走的）；容量满先 realloc 扩容。

1.2 删除第 i 位（取出，前移一位）

```
    p = L->elem + i - 1;  *e = *p;                      // p 指向被删元素
    for (++p; p <= L->elem + L->length - 1; ++p) *(p-1) = *p;  // 从前往后逐个前移
    --L->length;
```

↑ 前移必须**从前往后**；插入删除都是 O(n)，随机访问 O(1)。

核心记忆点

- 插入后移、删除前移，方向别搞反：空位在 i，后面的人往后让；删掉 i，后面的人往前补。
- 满了才 realloc：容量 += 增量，基址可能变（指针要重算）。

二、四种链表核心操作（只用 p、q 两指针）

约定：p 前 q 后；插入时 q 充当新结点；head 为头结点（数据域不用或作标记）。

2.1 通用准备：申请新结点

```
    q = (node*)malloc(sizeof(node));  q->data = x;
```

↑ malloc 返回 void* 要强转；申请一个 node 那么大；判空 exit(OVERFLOW)（全书惯例）。

2.2 单链表

头插　初始：p 指向 head

```
    q->next = p->next;  p->next = q;
```

↑ q 指向原 first，p 指向 q（改 2 根线）

尾插　初始：p 指向 head，先爬到最后

```
    while (p->next != NULL) p = p->next;   // 爬到尾：p->next 为空即是尾
    q->next = NULL;  p->next = q;
```

↑ 找尾 O(n)；q 作为新尾指向 NULL

中间插（第 i 个位置）　初始：p 指向 head，爬到 i-1 位置

```
    for (int j = 1; j < i; j++) p = p->next;   // p 停在第 i-1 个结点
    q->next = p->next;  p->next = q;
```

↑ p 在 i-1，把 q 插进 p 和 p->next 之间；定位前驱是插入删除的前提

跨越删除（值为 x）　初始：p 指向第一项数据，q 指向 head

```
    while (p != NULL && p->data != x) { q = p;  p = p->next; }   // q 跟着 p 爬
    q->next = p->next;  free(p);
```

↑ 循环结束时 q 是 p 的前驱；q->next 跨过 p 指向 p->next，释放 p

遍历　初始：p 指向第一项数据

```
    while (p != NULL) { /* 处理 p->data */  q = p;  p = p->next; }
```

↑ q = p; p = p->next; 始终保持 q 是 p 前驱

2.3 双向链表（多一根 prev，插入改 4 根线）

头插　初始：p 指向 head

```
    q->next = p->next;  q->prev = p;
    if (p->next != NULL) p->next->prev = q;   // 原 first 的 prev 改指 q
    p->next = q;
```

↑ 前驱方向两根 + 后继方向两根，共 4 根线；注意判断链表是否为空

跨越删除　初始：p 指向第一项数据，q 指向 head

```
    while (p != NULL && p->data != x) { q = p;  p = p->next; }
    q->next = p->next;  if (p->next) p->next->prev = q;  free(p);
```

↑ 双向删除要改两根线：q->next 跨过 p，且 p->next 的 prev 回指 q

2.4 单向循环链表（只把 NULL 换成 head）

尾插　初始：p 指向 head，爬到尾（循环条件不同！）

```
    while (p->next != head) p = p->next;   // 找尾条件是 != head
    q->next = head;  p->next = q;
```

↑ q 指向 head 保持循环
遍历/删除　循环条件 `p != head`（不是 p != NULL），其余同单链表，无需判空。

2.5 双向循环链表（head->prev 就是尾，尾插 O(1)！）

尾插　初始：p 指向 head->prev（O(1) 直接拿到尾！）

```
q->next = head;  q->prev = p;  p->next = q;  head->prev = q;
```

↑ 核心优势：head->prev 就是尾，4 根线 O(1) 完成；插入删除无需任何判空

2.6 对比表

链表类型	头插	尾插	中间插	删除
双向链表	4 根线	O(n) 找尾	4 根线	改两根线
单向循环	2 根线	O(n)，条件 != head	2 根线	q->next = p->next
双向循环	4 根线	**O(1)，head->prev 即尾**	4 根线	改两根线，无需判空

2.7 两个有序链表归并（双指针，谁小摘谁）——教材算法 2.21

约定：pa、pb 分别扫描 La、Lb；pc 指向结果尾（初值用 La 的头结点复用）。
初始：pa = La->next，pb = Lb->next，pc = La。

```
while (pa && pb)                          // 两个都没完才比较
    if (pa->data <= pb->data) { pc->next = pa;  pc = pa;  pa = pa->next; }
    else                      { pc->next = pb;  pc = pb;  pb = pb->next; }
pc->next = pa ? pa : pb;                  // 剩哪段接哪段
free(Lb);  return La;
```

↑ 三指针 pa/pb/pc 各管一件事：pc->next 挂小的，pc 跟上，小的前进；循环处理完后至多一段有剩余，直接接上。
核心记忆点

- 只有 p、q 两个指针：p 前（定位），q 后（新结点或前驱）。
- 插入两句：`q->next = p->next; p->next = q;`（双向再补 prev 两根线）
- 前进一步：`q = p; p = p->next;`　　跨越删除：`q->next = p->next; free(p);`
- 循环条件：单/双向链表判 != NULL，循环链表判 != head。
- 双向循环最划算：尾插 O(1)，操作不判空。
- 归并口诀：谁小摘谁，pc 永远跟在新尾后面。

三、栈与队列

3.1 顺序栈（top 指向下一空位）

约定：base 栈底（不变），top 栈顶（指向**下一空位**）。

```
*S->top++ = e;          // 入栈：先赋值，top 再++
*e = *--S->top;         // 出栈：先 --，再取值
```

↑ 空栈条件 `top == base`；栈满 `top - base == size`；realloc 后 top 要重算（基址搬了家）。
中序/表达式/括号匹配：凡是"后进先出地配对"，都是这套 Push/Pop + 一个"不配对就报错"的判断。

3.2 循环队列（牺牲一个单元区分空满）

约定：front 指向队头元素，rear 指向**下一空位**，容量 MAXQSIZE。

```
Q->base[Q->rear] = e;  Q->rear = (Q->rear + 1) % MAXQSIZE;   // 入队：放 rear，rear 模加
*e = Q->base[Q->front];  Q->front = (Q->front + 1) % MAXQSIZE; // 出队：取 front，front 模加
```

↑ 队空 `front == rear`；队满 `(rear+1) % MAXQSIZE == front`（绕一圈追上 front 前一格）。

3.3 链队列（头出尾进，两个指针）

```
Q->rear->next = p;  Q->rear = p;          // 入队：挂尾，尾指针跟上
p = Q->front->next;  Q->front->next = p->next;  // 出队：取头元素
if (Q->rear == p) Q->rear = Q->front;     // 删成空队列时尾指针收回
```

↑ 空队列条件 `Q->front == Q->rear`；出队删到空时 rear 必须收回，否则悬空。
核心记忆点

- 栈：先赋值后 ++（入），先 -- 后取值（出）。
- 循环队列一切取模；rear 永远追 front，牺牲一格防"满=空"。
- 链队列入队改尾指针，出队动头结点 next；删空收 rear。

四、串

4.1 短串 SStr（0 号单元存长度）

约定：`S.ch[0]` 存长度，串从 `S.ch[1]` 开始；定长 255，超长截断或报错。

```
T->ch[i+1] = chars[i];  T->ch[0] = i;  T->len = i;   // 赋值：拷内容、0 号存长、len 同步
```

4.2 BF 回溯 vs KMP 不回溯（对比表是灵魂）

	BF（算法 4.8）	KMP（算法 4.9）
主串扫描	可能扫多遍	只扫一遍，可边读文件边匹配
复杂度	O(n·m)	O(n+m)，但常数大，短模式不如 BF

KMP 主循环（背熟）：

```
while (i <= S.len && j <= T.len)
    if (j == 0 || S.ch[i] == T.ch[j]) { ++i; ++j; }   // j==0：模式从头再来
    else j = next[j];                                  // i 不动，j 沿 next 滑动
```

4.3 get_next：对模式串自己做一遍匹配（算法 4.10）

约定：i 扫描模式串，j 记当前最长前缀；`next[1] = 0`。

```
next[1] = 0;
while (i < T.len)
    if (j == 0 || T.ch[i] == T.ch[j]) { ++i; ++j;  next[i] = j; }  // 匹配：next[i+1] = j+1
    else j = next[j];                                              // 失配：j 继续滑动
```

↑ 和 KMP 主循环**长得一模一样**——求 next 就是拿模式串当主串、模式串当自己匹配；失配时 j 跳到 next[j] 继续比。

4.4 nextval：p_i == p_k 时直接继承（算法 4.11）

```
++i; ++j;
nextval[i] = (T.ch[i] != T.ch[j]) ? j : nextval[j];   // 相等则继承，省一次必败的比较
```

↑ 例："aaaab"：next = 0 1 2 3 4，nextval = 0 0 0 0 4。

核心记忆点

- KMP 三件套：主循环 `j==0` 或相等则双进，否则 `j=next[j]`；主串指针永远不回退。
- get_next 与 KMP 主循环同构，记住一个等于记住两个。
- nextval 只在 next 基础上加一句"相等就继承"。

五、数组与矩阵压缩

5.1 行主序地址公式（一个式子算所有维）

```
LOC(j1..jn) = 基址 + L * (c1*j1 + ... + cn*jn)
cn = L;  ci = L * b(i+1) * b(i+2) * ... * bn;   // 从最后一维往回累乘
```

↑ 三维 a[i][j][k]（4×5×6）：地址 = (i*5 + j)*6 + k = 30i + 6j + k。

5.2 特殊矩阵压缩（只记对称矩阵的 k）

```
k = i*(i-1)/2 + j - 1;    // 只存下三角：第 i 行前共有 i(i-1)/2 个元素
```

↑ i ≥ j 用上行；i < j 则交换 i、j 再代入。三角矩阵多存一个常数 c；对角矩阵按带宽逐行存。

5.3 三元组快速转置（算法 5.4，三步，背下来）

约定：num[col] = 第 col 列非零元个数；cpos[col] = 该列在 T 表中的起始位置。

```
for (t = 1; t <= M.tu; ++t) ++num[M.data[t].j];        // ① 统计每列几个
cpos[0] = 1;
for (col = 1; col < M.nu; ++col) cpos[col] = cpos[col-1] + num[col-1];  // ② 算每列放哪
for (p = 1; p <= M.tu; ++p) { q = cpos[M.data[p].j]++;  // ③ 一次扫描：按列定位放
    T->data[q] = {M.data[p].j, M.data[p].i, M.data[p].e}; }
```

↑ 普通转置是"按列反复扫全表"O(n·t)；快速转置先花两趟 O(nu+tu) 算出位置，第三趟一次到位。`cpos[col]++` 用完全列位置自动+1。

核心记忆点

- 压缩存储只问两件事：存哪个元素（下标公式）、放到数组第几个位置（cpos 公式）。
- 快速转置三步：数个数 → 算位置 → 一次放完。

六、广义表（递归三段式）

6.1 取表头/表尾

```
head = L->ptr.hp;    // 表头：第一个元素（原子或子表，都可能是）
tail = L->ptr.tp;    // 表尾：其余元素组成的表（一定是表）
```

6.2 求深度（算法 5.8，三行骨架）

```
if (!L) return 1;                  // 空表深度 1
if (L->tag == ATOM) return 0;      // 原子深度 0
for (max = 0, pp = L; pp; pp = pp->ptr.tp)   // 沿 tp 扫同层所有子表
    max = fmax(max, GListDepth(pp->ptr.hp));  // 对每个表头递归
return max + 1;
```

↑ 广义表递归三段式：① `if (!L)` 判空；② `tag` 分原子/子表两支；③ 子表沿 tp 扫同层、对 hp 递归。复制（算法 5.9）、建立（算法 5.10）全是这个骨架。

核心记忆点

- 表头可能是原子，表尾一定是表；空表深度 1、原子深度 0。
- n 个结点互不相似二叉树有 Catalan 数个：b(n) = C(2n,n)/(n+1)；n 结点树 t(n) = b(n-1)。

七、二叉树

7.1 三种遍历递归版（只差 Visit 的位置）

```
PreOrder(T):  Visit(T);  递归左;  递归右;    // DLR：根在最前
InOrder(T):   递归左;  Visit(T);  递归右;    // LDR：根在中间
PostOrder(T): 递归左;  递归右;  Visit(T);    // LRD：根在最后
```

↑ 口诀"先中后"指**根**的位置；左永远在右前。例：表达式树 a+b*(c-d)-e/f 得前缀 -+a*b-cd/ef、后缀 abcd-*+ef/-。

7.2 中序非递归（算法 6.5，一个 while 两个分支，必背）

约定：栈 S 模拟递归工作栈；p 当前结点。

初始：p 指向根。

```
while (p || !StackEmpty(S))
    if (p) { Push(S, p);  p = p->lchild; }        // ① 非空：进栈，继续向左
    else { Pop(S, p);  Visit(p->data);  p = p->rchild; }  // ② 空：弹栈访问，转向右
```

↑ 分支①一路压栈走到最左下；分支②弹出的就是"左子树已处理完"的结点；访问后转右子树重复。

7.3 按先序串建树（算法 6.7，'#' 是空树）

```
++i_;
if (pts[i_] == '#') return NULL;          // 空树
T->data = pts[i_];
T->lchild = CreateBiTree(pts);            // 递归建左
T->rchild = CreateBiTree(pts);            // 递归建右
```

↑ 全局 i_ 顺序扫描串；每个结点都是"造根→建左→建右"，'#' 返回 NULL 收拢。

7.4 线索二叉树（算法 6.10：遍历 + 改空指针 + 全局 pre）

约定：pre 全局变量，始终指向**刚访问过**的结点；空 lchild 改前驱线索，空 rchild 改后继线索。

```
void InThreading(BiThrTree p) {
    InThreading(p->lchild);                              // 左子树线索化
    if (!p->lchild) { p->LTag = Thread;  p->lchild = pre; }   // 我的空左 = 指向前驱
    if (!pre->rchild) { pre->RTag = Thread;  pre->rchild = p; } // 前驱的空右 = 指向我
    pre = p;                                             // pre 跟上
    InThreading(p->rchild);                              // 右子树线索化
}
```

↑ 只比中序遍历多三行：改前驱线索、改后继线索、pre 前进。找中序后继：右链是线索则直达；否则= 右子树最左下结点。

核心记忆点

- n 个结点的二叉链表必有 **n+1 个空链域**——线索化就是把它们利用起来。
- 线索树上遍历不需要栈，常数因子小。
- 回收二叉树：后序 free，返回 NULL，`T = FreeBiTree(T)` 必须接住（值参救不了实参）。

八、堆（完全二叉树按编号放数组）

约定：结点 i 的双亲 ⌊ i/2 ⌋，孩子 2i 和 2i+1；len/2 是最后一个分支结点。

8.1 向下筛选 SiftDown（算法 6.11，背熟）

前提：pos 的两棵子树都已是堆。约定：c 为左右孩子中较优先者。

```
while (pos <= H->len/2) {                // pos 超过 len/2 就是叶，停
    c = 2*pos;                           // 先取左孩子
    if (2*pos+1 <= H->len && Prior(H->elem[2*pos+1], H->elem[c])) c = 2*pos+1;  // 右更优先则换右
    if (Prior(H->elem[pos], H->elem[c])) break;   // 已满足堆序，收工
    swap(H, pos, c);  pos = c;           // 与较优孩子交换，继续往下
}
```

↑ 每轮干两件事：选较优孩子、不满足就下沉一层。O(log n)。

8.2 插入 = 加到堆尾后向上冒（算法 6.12）

```
H->elem[++H->len] = e;  curr = H->len;               // 先放堆尾
while (curr != 1 && !Prior(H->elem[curr/2], H->elem[curr])) {   // 双亲不较优先
    swap(H, curr, curr/2);  curr /= 2;               // 交换，curr 上移
}
```

↑ 与 SiftDown 方向相反：从叶向上。O(log n)。

8.3 删堆顶（算法 6.13，三步）

```
e = H->elem[1];                     // ① 取出堆顶
H->elem[1] = H->elem[H->len--];     // ② 堆尾顶上堆顶，长度减 1
SiftDown(H, 1);                     // ③ 从堆顶向下筛选
```

8.4 建堆 MakeHeap（算法 6.14，一句话）

```
for (i = H->len/2; i > 0; --i) SiftDown(H, i);   // 从最后一个分支结点向前逐个筛选，O(n)
```

核心记忆点

- 完全二叉树编号公式 i/2、2i、2i+1 是堆一切操作的地基。
- 两方向：向下 SiftDown（删堆顶/建堆），向上冒（插入）。
- 大顶堆小顶堆只换一个优先函数 `Prior`，结构代码零改动。
- 建堆从 len/2 往前，O(n) 不是 O(n log n)。

九、赫夫曼树与编码

9.1 构造（算法 6.15：小顶堆存森林，反复取两个最小合并）

```
while (H->len > 1) {
    lt = DelTop(H);  rt = DelTop(H);          // 取权值最小的两棵树
    root->weight = lt->weight + rt->weight;   // 新根权 = 两者之和
    root->lchild = lt;  root->rchild = rt;    // 接上左右
    Insert(H, root);                          // 新树插回堆
}
```

↑ 循环 n-1 次，每次 DelTop×2 + Insert，O(n log n)；本质就是"每次合并最小的两堆"。

9.2 生成编码（算法 6.16：先序遍历 + cd[] 记录路径）

```
if (x->data != '*') { 保存 cd 为该字符编码;  return; }   // 到叶子 = 拿到一个字符的编码
cd[cdlen] = '0';  GenHFCodes(x->lchild, cd, cdlen+1, HC);  // 向左压 0
cd[cdlen] = '1';  GenHFCodes(x->rchild, cd, cdlen+1, HC);  // 向右压 1
```

↑ cd[] 像一个栈：递归到叶时，cd 里就是从根到该叶的路径 01 串。

核心记忆点

- 赫夫曼编码 = 不等长 + 前缀编码（任何编码不是别的前缀）+ 权值大的路径短。
- 译码：从根按 0/1 走左右，遇叶出字符回根。

十、并查集（双亲数组，下标即成员）

约定：`nodes[i].parent`：负数 = 根（绝对值是子集大小），非负 = 双亲下标。

10.1 三行查找、两行合并

```
while (S->nodes[i].parent > 0) i = S->nodes[i].parent;   // Find：顺双亲链爬到根（算法 6.20）

S->nodes[i].parent = j;                                   // Union：一根指向另一根（算法 6.21）
```

10.2 加权合并 Union_wt（算法 6.22：小集挂大集）

```
if (S->nodes[i].parent <= S->nodes[j].parent) {  // 注意负数比大小：|Si| >= |Sj|
    S->nodes[i].parent += S->nodes[j].parent;    // i 的计数累加 j 的
    S->nodes[j].parent = i;                       // j 挂到 i
}  // 否则对称处理
```

↑ 根的 parent 存 -成员数；比较"<= "就是比较绝对值大小；树深 ≤ ⌊ log n⌋ +1。

10.3 路径压缩 Find_pc（算法 6.23：查完把沿途全挂到根）

```
root = i;
while (S->nodes[root].parent > 0) root = S->nodes[root].parent;   // 第一趟：找根
while (S->nodes[i].parent != root) { p = S->nodes[i].parent;  S->nodes[i].parent = root;  i = p; }
```

↑ 第二趟把 i 到根路径上每个结点的 parent 直接改成 root；m 次操作总复杂度近似 O(m)（α(n) ≤ 4）。

10.4 对比表

版本	查找	合并	备注
+ 加权	O(log n)	O(1)	小挂大
+ 路径压缩	近似 O(1)	同左	Find 里顺手压缩
用途	等价类划分、Kruskal 判回路		

核心记忆点

- 负数为根、绝对值计数；查找就是 while (parent > 0) 爬。
- 加权：小挂大；压缩：沿途直挂根。两招都用上就是"并查集完全体"。

十一、图

11.1 建邻接表（算法 7.3：逐弧头插入弧尾的链表）

```
G->vexs[i].firstArc = NULL;                    // 初始化：每个顶点的链表清空
/* 对每条弧 (u → v)：  */
p->adjvex = v;                                  // 弧结点记住弧头下标
p->nextArc = G->vexs[u].firstArc;  G->vexs[u].firstArc = p;   // 头插进 u 的链表
```

↑ 邻接表 = 顶点数组 + n 条单链表；有向图只存"出弧"，无向图同一条边存两次（i→j 和 j→i）。入度 = 全表扫一遍数 adjvex==v 的个数（或建逆邻接表/十字链表）。

11.2 DFS / BFS 骨架（算法 7.5 / 7.6，同一个外壳）

```
for (v = 0; v < G->n; ++v) visited[v] = 0;                  // ① 标志数组清零
for (v = 0; v < G->n; ++v)
    if (!visited[v]) DFS(G, v);   /* 或 BFS(G, v) */        // ② 每个连通分量调一次

void DFS(G, v) {                                            // 深：递归钻到底
    visited[v] = 1;  Visit(v);
    for (w = FirstAdjVex(G,v); w >= 0; w = NextAdjVex(G,v))
        if (!visited[w]) DFS(G, w);
}
void BFS(G, v) {                                            // 宽：队列一层层扩散
    visited[v] = 1;  Visit(v);  EnQueue(Q, v);
    while (!QueueEmpty(Q)) {
        DeQueue(Q, u);
        for (w = FirstAdjVex(G,u); w >= 0; w = NextAdjVex(G,u))
            if (!visited[w]) { visited[w] = 1;  Visit(w);  EnQueue(Q, w); }
    }
}
```

↑ 邻接表上都是 O(n+e)；区别只在 DFS 用递归栈、BFS 用队列。②的外层 for 保证非连通图每个分量都被访问。

11.3 Prim 与 Dijkstra 是同一个框架（对比表必背）

约定：U = 已入选顶点集；辅助数组记 V-U 各顶点的"当前最优"，每次选最小、入选后刷新。

	Prim（算法 7.9，最小生成树）	**Dijkstra（算法 7.12，单源最短路）**
初始化	closedge[j] = w(u, j)，源点 u	Dist[j] = w(源, j)
每轮	选最小 lowcost 顶点 k 入选	选最小 Dist 顶点 k 确定
刷新	用 k 的行：w(k,j) < lowcost[j] 则更新	用 k 的弧：Dist[k]+w < Dist[j] 则更新
复杂度	邻接矩阵 $O(n^2)$；堆+邻接表 O((n+e)log n)	同上

Prim 核心骨架：

```
for (i = 1; i < G->n; ++i) {
    选 closedge 中最小 lowcost 的顶点 k;       // 小顶堆可优化
    输出边 (k, closedge[k].adjIdx);  closedge[k].lowcost = 0;   // k 入选
    for (j = 0; j < G->n; ++j)                 // 用新入选的 k 刷新所有顶点
        if (G->arcs[k][j] < closedge[j].lowcost) { closedge[j].adjIdx = k;  closedge[j].lowcost = G-
>arcs[k][j]; }
}
```

↑ 记住四个动作：**选最小 → 入选 → 沿 k 刷新 → 重复 n-1 次**。Dijkstra 把"边权"换成"路径和"再顺手记个前驱 prev 即是。

11.4 Kruskal（算法 7.10：排序 + 并查集判回路）

```
边按权升序排好（qsort / 小顶堆）;
for (每条边 (u, v, w) 按序) {
    if (Union_wt(S, u, v)) { 加入 T;  ++cnt; }   // 分属不同子集 = 不成环，收下
    if (cnt == n-1) break;                        // 收满 n-1 条收工
}
```

↑ 若两端已同属一个子集则加边必成环，放弃；O(e log e)，适合稀疏图。

11.5 Floyd（算法 7.13：一句话 + 三重循环）

```
for (k = 0; k < n; ++k)              // k 必须放最外层：逐个点"放行"当中转
    for (i = 0; i < n; ++i)
        for (j = 0; j < n; ++j)
            if (D[i][k] + D[k][j] < D[i][j]) D[i][j] = D[i][k] + D[k][j];
```

↑ D 从邻接矩阵出发；k 放外层是"允许前 k 个顶点作中转"的递推含义。

11.6 拓扑排序（算法 7.14：入度数组 + 零入度队列）

```
for (每条弧 i→j) ++indeg[j];                    // ① 统计入度
入度为 0 的顶点入队 Q;
while (出队顶点 i) {
    输出 i;  ++count;
    for (i 的每条出弧 i→j) if (--indeg[j] == 0) EnQueue(Q, j);   // ② 删弧 = 入度减 1
}
count < n 即存在环;
```

↑ 全程 O(n+e)；输出顶点数 < n 就是 AOV 网有环（工程无法进行）。

11.7 关键路径（算法 7.15/7.16：拓扑序两趟）

```
拓扑排序同时顺推：ve[j] = max(ve[j], ve[i] + w(i,j));          // 事件的最早发生时间
按逆拓扑序回推：  vl[i] = min(vl[i], vl[j] - w(i,j));          // 事件的最迟发生时间（vl 初值 = ve[汇点]）
每条弧 <i,j>：ae = ve[i];  al = vl[j] - w;  if (ae == al) → 关键活动
```

↑ 四个量两两一对：事件的 ve/vl（拓扑/逆拓扑推），活动的 ae/al（由事件量算）；ae=al 的活动串成关键路径。关键路径可能不止一条。

11.8 二部图最大匹配——匈牙利（算法 7.18，递归四句）

约定：`matchTo[v]` = V 集顶点 v 匹配的 U 集顶点（0 = 未匹配）；visited 每轮清零。

```
for (u 的每个邻接点 v) {
    if (visited[v]) continue;  visited[v] = 1;          // 本轮试过就不再试
    if (matchTo[v] == 0 || dfs(matchTo[v])) {           // v 空 或 v 的原配能让位
        matchTo[v] = u;  return 1;                      // 匹配成功（异或翻转）
    }
}
return 0;
```

↑ 精髓就一句：`matchTo[v]==0 || dfs(matchTo[v])`——已匹配的顶点递归"让位"，沿增广路径翻转。主函数对每个 u 调一次，O(n·e)。

核心记忆点

- 邻接表建表 = 逐弧头插；DFS/BFS 共用"标志数组 + 逐分量调用"外壳。
- Prim/Dijkstra 同框架：辅助数组记当前最优，选最小→入选→刷新。
- 拓扑排序：入度减 1 就是删弧；Floyd：k 必在最外层。
- 关键路径：ve 顺推取 max，vl 逆推取 min，ae==al 是关键活动。
- 匈牙利：v 没人要直接收；有人要就递归让位。

十二、回溯法通用模板（教材 6.8）

```
void Trial(int i) {
    if (i > n) { 输出一个解;  return; }          // 到底 = 得到一个完整解
    for (每个可选选择 j) {
        做选择;                                   // 试探
        if (合法) Trial(i + 1);                   // 合法才深入，否则剪枝
        撤销选择;                                 // 回溯：恢复原状
    }
}
```

↑幂集（取/舍）、四皇后（放棋子/判冲突）、迷宫、骑士游历全是这一个模板：递归 + 循环试探 + 合法性剪枝 + 撤销。
核心记忆点

- 回溯 = 在状态树上做带剪枝的先序遍历；三个固定动作：试探 → 深入 → 撤销。
- 恢复现场（撤销）和试探必须成对出现，否则影响兄弟分支。

附：全书口诀速记（一页背完）

1. 链表：p 前 q 后；插入 `q->next=p->next; p->next=q;`；前进 `q=p; p=p->next;`；删除 `q->next=p->next; free(p)`。
2. 栈：`先赋值后++` 入、`先--后取值` 出；循环队列全取模、rear 追 front 牺牲一格。
3. KMP：失配 `j=next[j]`，i 永不回退；get_next 与主循环同构；nextval 相等就继承。
4. 快速转置：数个数 → 算位置 → 一次放完。
5. 遍历：根的位置定先中后；非递归中序 = `if(p)进栈向左 else 弹栈访问转右`。
6. 线索化 = 中序遍历 + 三行（改前驱、改后继、pre 跟上）。
7. 堆：i 的双亲 i/2、孩子 2i/2i+1；向下 SiftDown、向上冒；建堆从 len/2 起。
8. 并查集：负数为根绝对值计数；小挂大、沿途直挂根。
9. 图：Prim/Dijkstra 一框架（选最小→入选→刷新）；拓扑入度减 1；Floyd k 在外层；关键路径 ve 取 max、vl 取 min、ae==al。
10. 回溯：试探 → 合法才深入 → 撤销恢复。